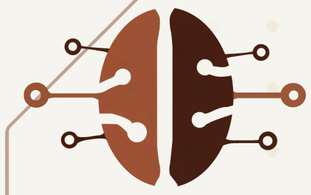
Extreme AI engineering

Abrazando el cambio ...a base de tokens

22/10/2026 Roberto Sócrates





¿A qué venimos?

Prompt engineering
Context engineering
Agent engineering
Harness engineering
Loop engineering
Graph engineering

No venimos a hablar de...

Vibe coding

Quiero que hagas... "una presentación sobre IA"

El mundo ha cambiado



Galadriel (Cate Blanchett, New Line Cinema: Las dos torres, 2002)

¿Cómo lo afronta la industria?

2006 Google

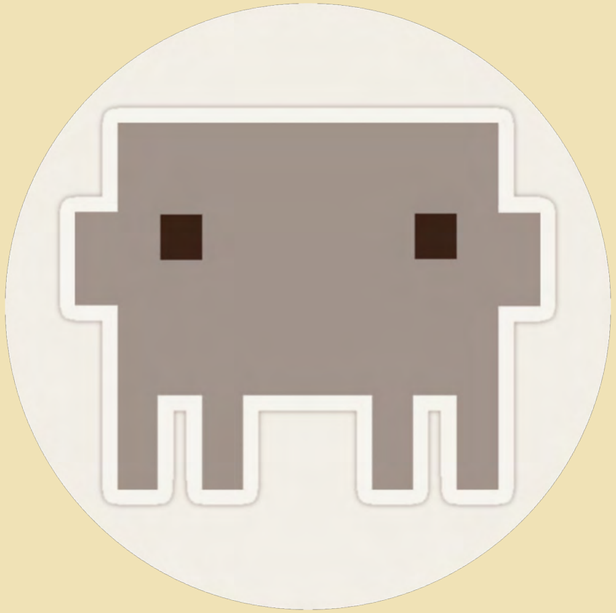
2006 Google

2016 Stack Overflow

2016 Stack Overflow

2026 Claude Code

a. C. / d. C.



¿Cómo lo afrontamos nosotros?

Necesitamos un mapa



…y retomar el timón



Logo: Kubernetes® (cncf/artwork), CC BY 4.0
Marca registrada de The Linux Foundation; sin afiliación ni patrocinio.

Disclaimer

Aprendiz de todo, maestro de nada

Esta información se proporciona tal cual, on an «as-is» basis, sin ofrecer garantías ni asumir responsabilidad por su buen o mal uso

La herramienta

La herramienta

¿Qué conocemos de ella?

Inside out

El modelo
El arnés
La UI
El usuario

La herramienta

El modelo

El modelo

Tiene conocimiento de casi todo

es casi omnisciente

porque lo ha leído casi todo

y lo guarda comprimido

Delétang et al. (Google DeepMind), Language Modeling Is Compression (ICLR 2024)



Foto: David Iliff, CC BY-SA 4.0 https://commons.wikimedia.org/w/index.php?curid=42693401


HISTORIAS
CUENTOS
SABIDURÍA
AVENTURAS
HISTORIAS
AVENTURAS
ZIP


HISTORIAS
CUENTOS
SABIDURÍA
AVENTURAS
HISTORIAS
AVENTURAS
JPEG

Criterio y decisión

Lo sabe casi todo,
pero no sabe lo que queremos

La decisión de lo que queremos aún es nuestra
Y para decidir hay que poner nuestro criterio

La herramienta

El arnés

El arnés

En medio,
con al menos un bucle hacia cada lado

hacia la UI nos atiende, pregunta y contesta
hacia el modelo lo invoca y ejecuta lo que le indica

Anthropic, Claude Code, How Claude Code works (2026)

Garantía de cumplimiento

Quizá — **Se lee**

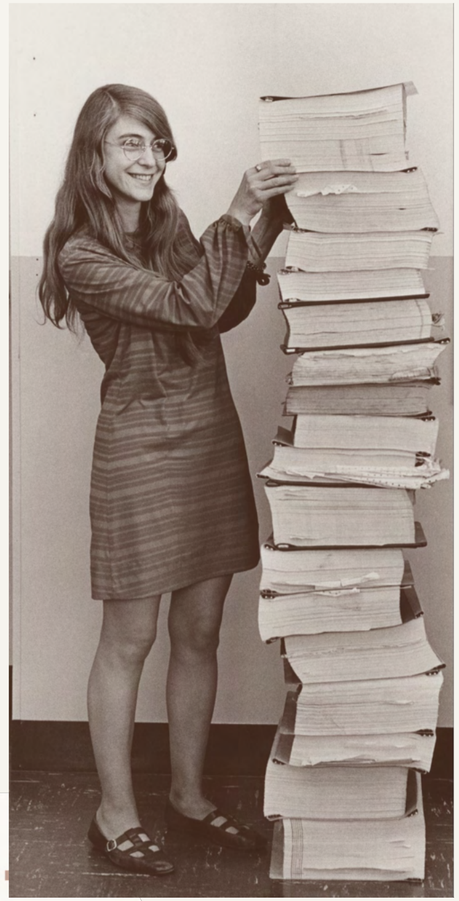
memoria · skills · prompt · output style

Siempre — **Se ejecuta**

herramientas · permisos · hooks

1968

Margaret Hamilton y el P01



1936-2026

La nota se lee
La salvaguarda se ejecuta

simulador: Lauren lanza el P01 en vuelo y lo tira
NASA y MIT: no hace falta código, basta una nota
Apolo 8: Lovell lanza el P01 y pierde la alineación
después: la salvaguarda se implementa en código

Margaret Hamilton, software de vuelo del Apolo (1968)

Lo que se ejecuta

- **Internas**
  - Integradas (AskUserQuestion) y embebidas (find, grep)
- **Conectividad externa**
  - MCP, navegador y WebMCP
- **Seguridad**
  - Permisos y sandbox
- **Abierto a la extensión**
  - Hooks

Anthropic, Claude Code, Tools reference y Extend Claude Code (2026)

Clasificadores y validadores

Momento · Naturaleza · Resultado

momento: antes de actuar impide; después, corrige
naturaleza: determinista o con un modelo
resultado: garantizado, o acierta casi siempre

Birgitta Böckeler, Harness engineering for coding agent users (2026)

Los hooks

Principio Hollywood

callback: en el prompt, la herramienta o el fin de turno
interceptor: puede vetar, antes de actuar o de acabar
aspecto: un matcher lo aplica a muchas herramientas

Anthropic, Claude Code, Hooks reference (2026)

El agente main y sus subagentes

Un FutureTask,
sin memoria compartida

en primer plano bloquea *get()*
en segundo plano avisa al acabar *done()*
con su propio contexto y solo devuelve el resultado

Anthropic, Claude Code, Create custom subagents (2026)

¿Un hook que invoca un agente?

Un subagente como validador

Verify that all unit tests pass

Run the test suite and check the results

Anthropic, Claude Code, Hooks reference (2026)

2025

Ralph Wiggum loop

Un loop del mismo prompt hasta cumplir el objetivo

El validador era el agente, sin garantía

...el plugin de Claude Code solo garantiza el bucle

Geoffrey Huntley, Ralph Wiggum as a «software engineer» (2025)

2026

Nuestros propios loops

«My job is to write loops»

Modelar el flujo de los agentes dentro de esos loops

Boris Cherny, Acquired Unplugged (WorkOS, 2026)

Invocando el modelo

Stateless, no recuerda nada
Con caché, se lo enviamos todo

lo que lee: instrucciones, memoria y conversación
lo que puede pedir: herramientas o la entrega

Anthropic, Using the Messages API (2026)

Lo que se lee, el contexto

- **Memoria**
  - Usuario, proyecto y sesión
- **Skills**
  - Solo la cabecera
- **Prompt**
  - Más allá del ultrathink
- **Output style**
  - ¿Qué tal en html y css?

Perla

Lo que no se carga no existe

La poda de CLAUDE.md y MEMORY.md

De MEMORY.md solo se cargan 200 líneas o 25 KB
CLAUDE.md: menos de 200 líneas, o se cumple peor

Anthropic, Claude Code, How Claude remembers your project (2026)

REVIEW
PENDING

Un contexto que se construye navegando

Un grafo de entidades
No un conjunto de CLAUDE.md enormes

dimensiones: intención (north star), principios, criterios
hacia arriba: el porqué, de la slide a la intención
hacia abajo: el qué, de la intención a la slide

La herramienta

¿Cómo le hablamos a la UI?

Nuestro propio idioma como lenguaje de programación



"The hottest new programming language is English" (Andrej Karpathy, 2023)
Fotogramas: Star Trek IV: The Voyage Home (Paramount, 1986)

Seis perlas para los prompts

1. Objetivo primero
2. Contexto en U y duplicado
3. Que te pregunte
4. Lealtad a la verdad
5. Resumen ejecutivo
6. Output style

P01

Objetivo primero

Empieza el prompt por el para qué
Finaliza con el cómo *(en caso necesario)*

El genio concede al pie de la letra:
*Si empiezas por el **cómo**, optimiza el cómo*

P02 Contexto en U y duplicado

Lo importante, al principio y al final

El material, en medio

El modelo atiende peor al centro de un contexto largo

Modelos no razonadores: *pega la petición dos veces*

Nelson F. Liu et al., Lost in the Middle (2023)
Leviathan, Kalman y Matias (Google Research), Prompt Repetition Improves Non-Reasoning LLMs (2025)

P03

Que te pregunte

En cualquier fase, le puedes pedir

¡que te pregunte!

Interview me in detail using the AskUserQuestion tool

Anthropic, Best practices for Claude Code (2026), «Let Claude interview you»

P04

Lealtad a la verdad

Pídele lealtad a la verdad *(franqueza)*
y que distinga lo comprobado de lo supuesto

*Por su entrenamiento, **los modelos** tienden a darnos la razón*
Reduce el sesgo, no lo elimina: sigue siendo contexto

Mrinank Sharma et al. (Anthropic), Towards Understanding Sycophancy in Language Models (2023)

P05

Resumen ejecutivo

Pídele resúmenes ejecutivos

con lo que ha actualizado y el estado actual

Hallazgos
Decisiones y su motivo
Supuestos y cuestiones abiertas
Puntos de acción

P06

Output style

El formato de la salida puede ser muy rico

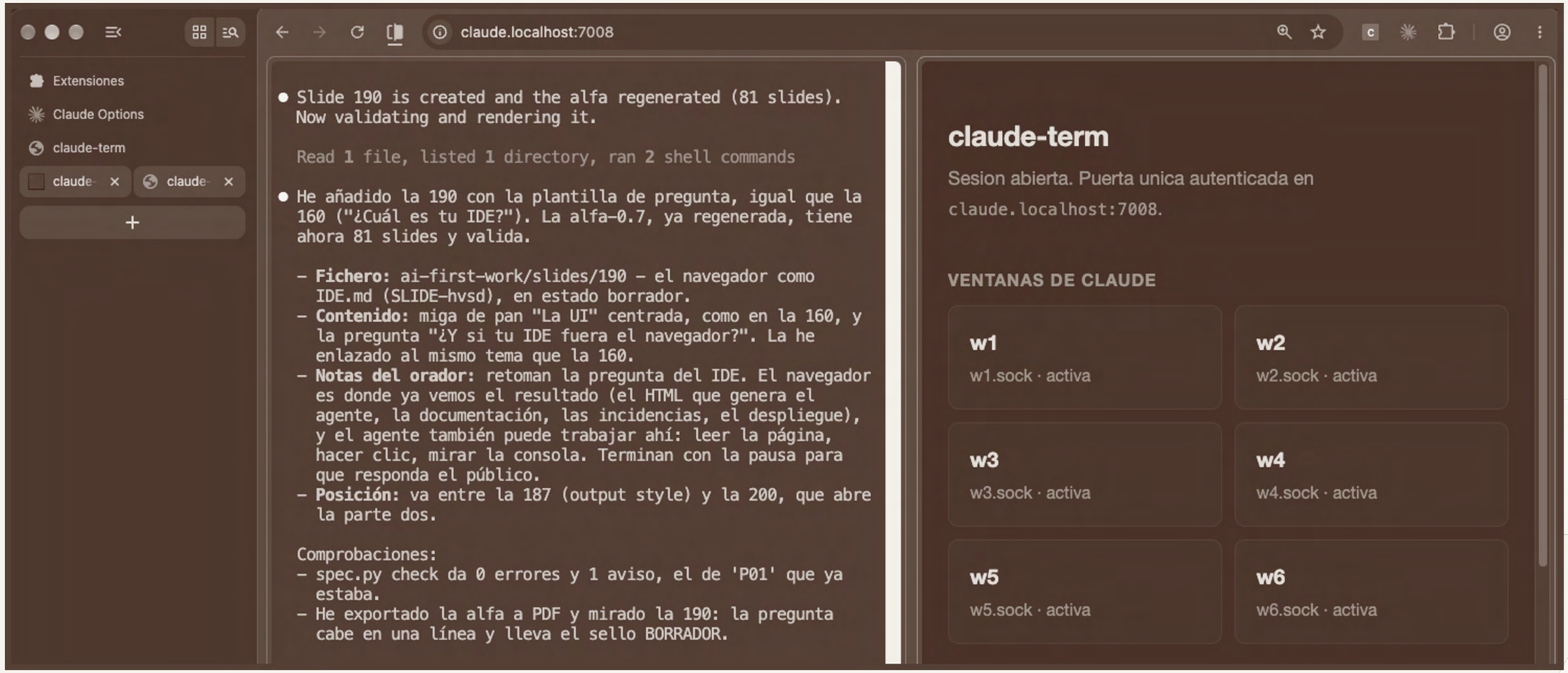
Múltiples dimensiones: **HTML** y **ASD-STE100**
Es una instrucción en el contexto, **sin garantía de ejecución**

Anthropic, Claude Code, Output styles (2026)
ASD-STE100, Issue 9 (2025): https://www.asd-ste100.org/assets/files/ASD-STE100_ISSUE9.pdf

El usuario

¿Y si el mejor IDE fuera tu navegador?

Tus propias herramientas



Nuestros datos

1973

Del procedimiento al dato

Hoy navega el agente

Bachman: el dato al centro, y el programador navega por él
Smalltalk: no recorras el dato, mándale un mensaje
*The big idea is **"messaging"** (Kay)*

Charles W. Bachman, The Programmer as Navigator (1973). Alan Kay, correo a la lista squeak-dev

1970 El acceso declarativo al dato

Modelar los datos, *no navegarlos*
El timón es nuestro, *declaramos el qué*

"...protected from having to know how the data is organized in the machine"

delegamos la navegación

E. F. Codd, A Relational Model of Data (1970)

1992

Especificarlo todo

Verba volant, scripta manent

Especificación en el repo, junto al código

Documentación estructurada para el arnés y el modelo

Cada concepto dentro de su workflow con su grafo

Jack W. Reeves, What Is Software Design? (1992)

Un formato y una ontología

OKF más nuestra ontología

formato: un Markdown con cabecera YAML por concepto, enlazados

ontología: *instancias de clases*, ids estables y relaciones

Google Cloud, Open Knowledge Format (2026)

1974

Separación de asuntos

Un fichero .md para cada concepto

Dijkstra: no pensar en todo a la vez

Principio de responsabilidad única: el frontmatter es la instancia

...y su clase, **la ontología**

Edsger W. Dijkstra, On the role of scientific thought (EWD447) (1974)

REVIEW
PENDING

Perla

El gobierno de la decisión

Ayudar al modelo

Para que nos ayude a nosotros

intent y **constitution**: para qué y con qué principios

decision rules y **methodology**: cómo y quién decide (clasificadores)

quality gates: validan invariantes y criterios de salida

Las fases del proyecto

2025

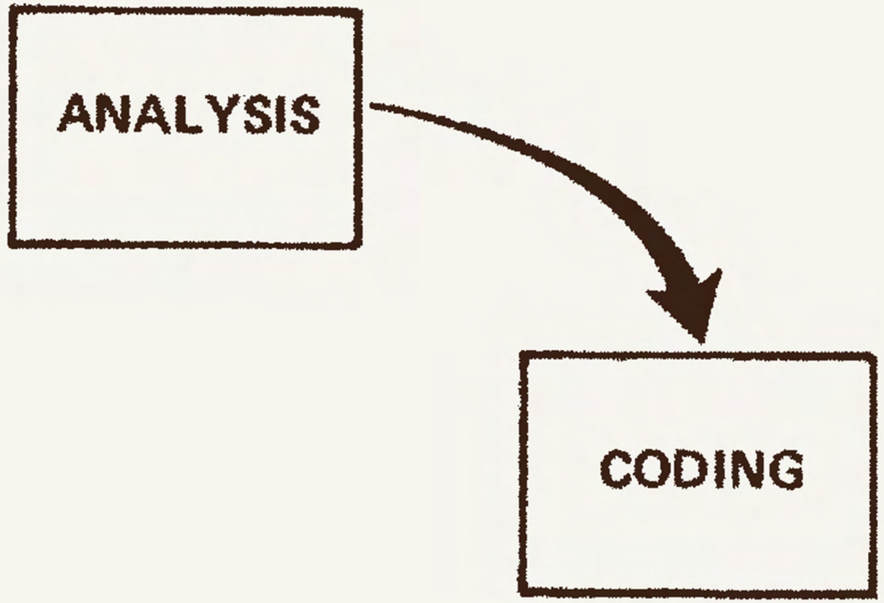
Spec-driven y RPI

investigar > especificar > planificar > partir en tareas > implementar > validar

¿un waterfall a base de quemar tokens?

GitHub Spec Kit (2025); Dex Horthy, HumanLayer (2025); Birgitta Böckeler, Thoughtworks (2025)

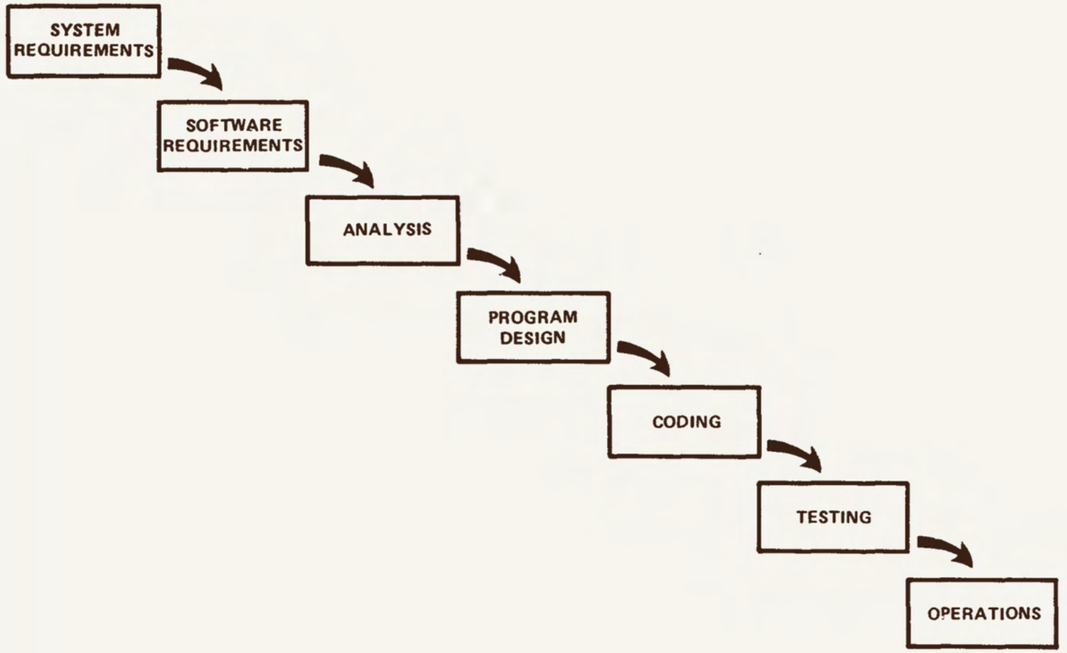
1970 Waterfall y el paper mal entendido



All that is required if the effort is sufficiently small

Winston W. Royce, Managing the Development of Large Software Systems (1970), p. 328

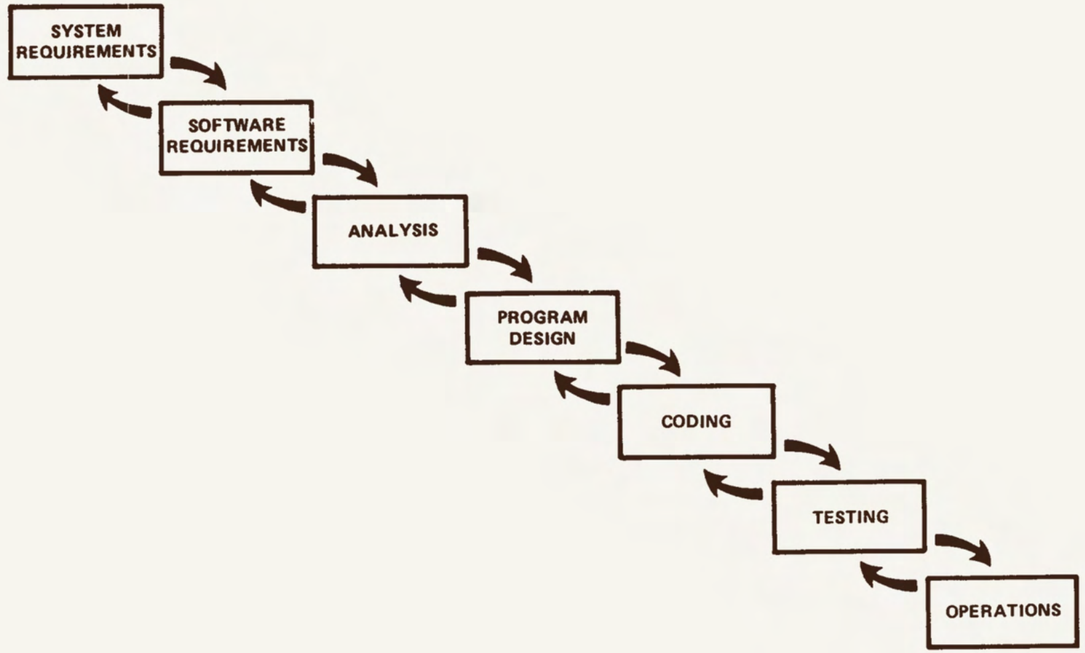
1970 Waterfall y el paper mal entendido



Risky and invites failure

Winston W. Royce, Managing the Development of Large Software Systems (1970), p. 329

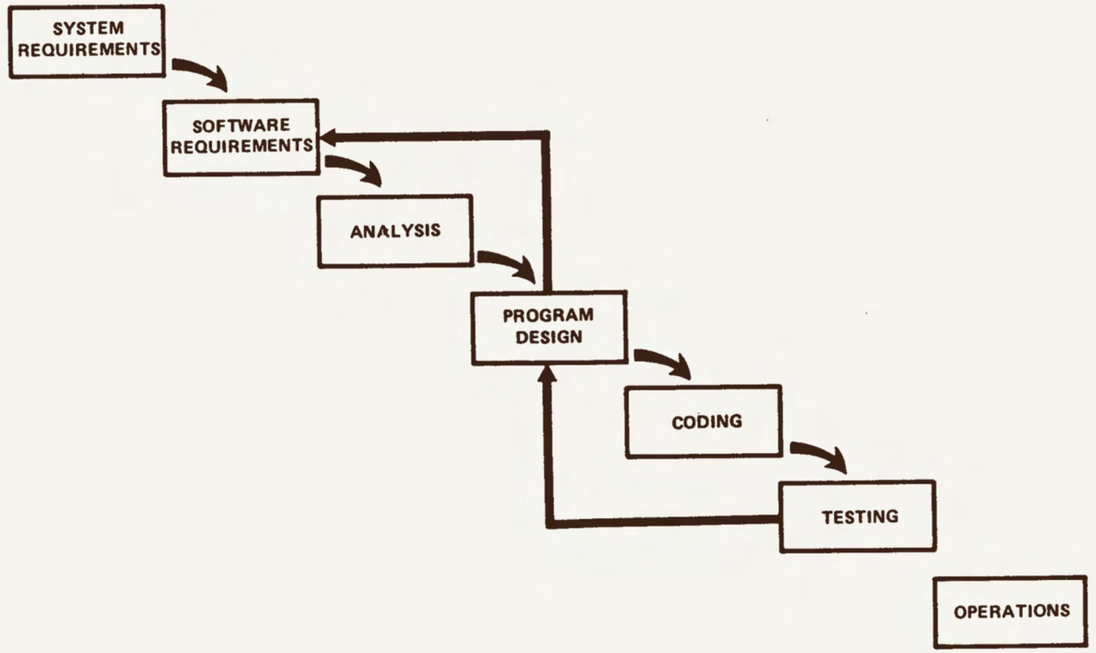
1970 Waterfall y el paper mal entendido



Hopefully [...] confined to successive steps

Winston W. Royce, Managing the Development of Large Software Systems (1970), p. 330

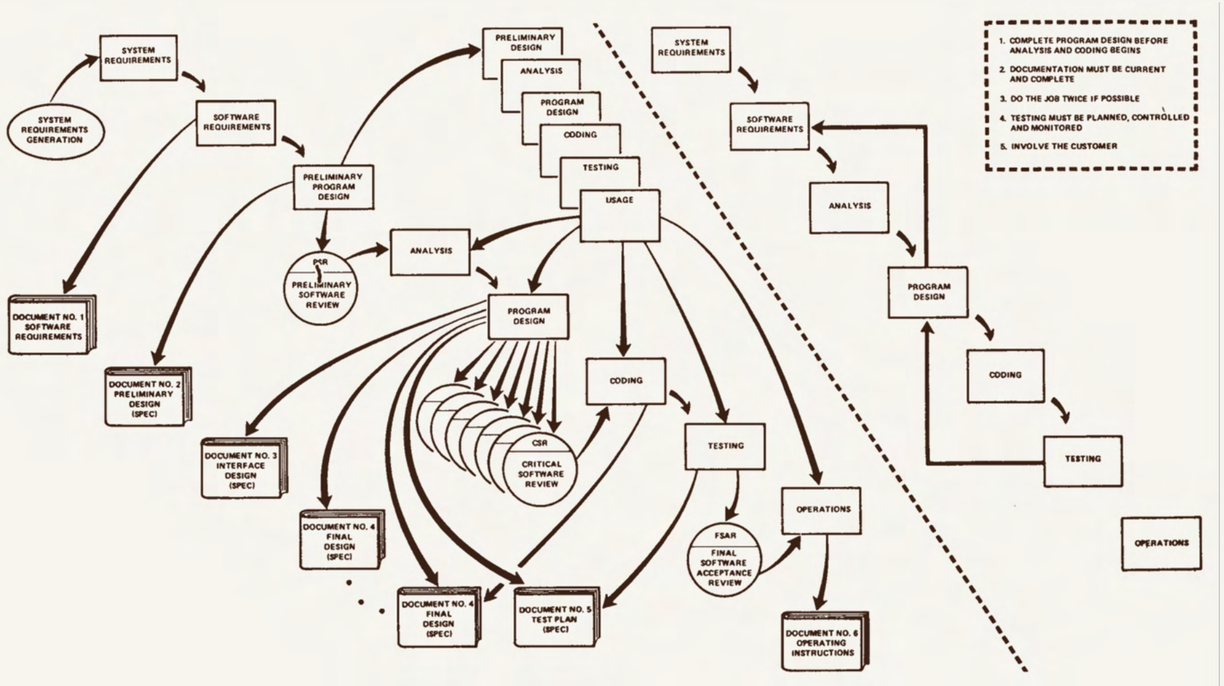
1970 Waterfall y el paper mal entendido



Unfortunately, [...] the design iterations are never confined to the successive steps

Winston W. Royce, Managing the Development of Large Software Systems (1970), p. 330

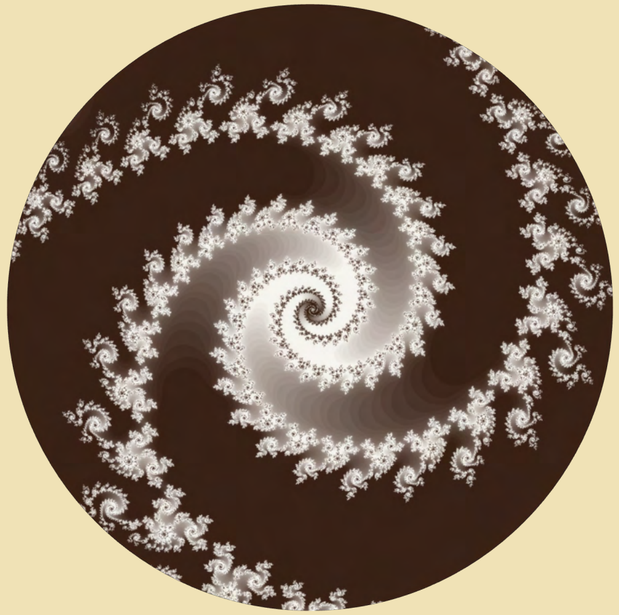
1970 Waterfall y el paper mal entendido



[...] the five steps that I feel necessary to transform a risky development process

Winston W. Royce, Managing the Development of Large Software Systems (1970), pp. 335 y 338

La magia de iterar



2021

Método científico y descubrimiento

Usar un enfoque empírico y científico para resolver problemas prácticos de software

Royce experimentó una vez
Nosotros, en múltiples iteraciones
(los agentes y yo)

Dave Farley, Modern Software Engineering (2021), citado en Tech Lead Journal, episodio 100 (2022)

Todo proyecto tiene una ontología

tareas
preguntas abiertas
casos de uso
criterios de aceptación
iteración
estados
flujos

Ejemplo

Cada concepto en su fichero

iteraciones/alpha-0-8.md

```
---
type: Iteración
estado: cerrada
tag: alpha-0.8
---
# alpha-0.8
```

preguntas/publico.md

```
---
type: Pregunta
estado: resuelta
---
# Público

Confirmado por el autor:
desarrolladores del ecosistema
Java, seniority mixta, que ya
usan IA a diario.
```

Iteraciones, tareas, dudas…

con un estado de valores cerrados

iteración alpha-0.8: estado cerrada, tag alpha-0.8
pregunta "Público": estado resuelta
y el resto de la literatura en el cuerpo

1999

Un id por entidad

Cada entidad es única

y la referenciamos por su id

@Tag("CRITERION-kqzt")

Customers write tests demonstrating that features are finished

Kent Beck, Extreme Programming Explained (1999), cap. 10

Perla

Git, siempre Git

Al menos repo local

Hook para tener un commit tras cada respuesta suya

Código y especificación, juntos

Tags para los hitos. **Ramas** para *experimentar*

Concurrencia: worktrees o branches

¿Hace falta reinventar la rueda?



Tan sólo hemos de facilitarle a los agentes manejar el flujo, modelándolo para cada proyecto

La escasez se desplaza

Una gran cantidad de información crea una pobreza de atención

Tenemos que protegernos de la carga cognitiva innecesaria

Herbert A. Simon, «Designing Organizations for an Information-Rich World» (1971), pp. 40-43

¿Codificar es un arte?

¿o una habilidad?

Las cuatro reglas de Beck

1. Pasa los tests
2. Revela la intención
3. Sin duplicación
4. Mínimo de elementos

Martin Fowler, BeckDesignRules (2015)

R01

Uncle Bob ya no lee el código

...pero sí verifica todo

Rodea a sus agentes de **restricciones extremas**
"El código limpio sigue importando"
(los agentes se confunden como nosotros)

Robert C. Martin, en X (23 y 25 de julio de 2026)

R01 Red, green, refactor, *pero desde ATDD*

Un test se ejecuta
Una instrucción para el LLM, quizás no

El test debe ser reflejo de la especificación

(evita que el genio no haga trampas)

Kent Beck, Test Driven Development: By Example (2002)

R02 Modularidad para una atención finita

1972

Un módulo cada vez

Parnas: "study the system one module at a time"

2025

Attention budget

El contexto del agente también se agota:
cuanto más se llena, peor atiende

Parnas (1972) · Anthropic (2025)

R03

Don't repeat yourself!

Una regla que habla del conocimiento

Cada decisión del sistema tiene que estar escrita en un solo sitio

Vale igual para la especificación y los prompts

Andy Hunt y Dave Thomas, The Pragmatic Programmer (1999)

R04

YAGNI

Producir es barato
Mantener, no tanto
(quema tokens y programadores)

El modelo añade capas sin pedirlas
El modelo no parará de proponer *si le das cuerda*

Ron Jeffries, You're NOT gonna need it! (1998)

¿Clean code y SOLID?

Never send an LLM to do the job of a linter or a structural test

El diseño: *en el formateador, el linter y ArchUnit*
La validación: *en un hook*
La deuda: *¿en el quality gate del CI?*

HumanLayer (2025) · OpenAI, Harness engineering (2026)

Codificar es aplicar más que habilidades

¿Cuánta deuda técnica se está generando?

2023

Features and futures

Producir **features** es barato
Preservar **the future** requiere acción

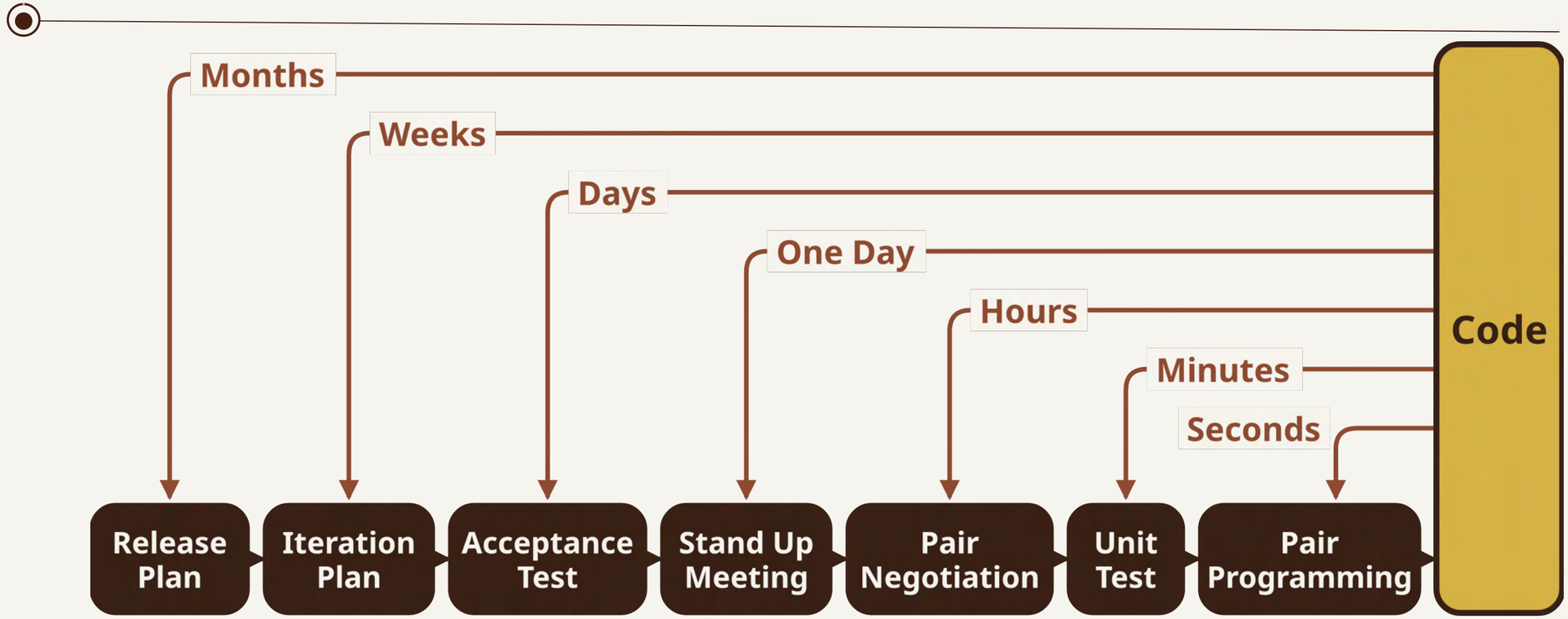
El **refactor** es una **inversión**
*(después del **green** con un **loop** con entidad propia)*

Kent Beck, Tidy First? (2023) · Third Loop, ep. 2 (2026)

Loops and graphs

2001

XP ya eran loops



Varias escalas a la vez, y todas vuelven al código

Don Wells, Planning/Feedback Loops (2001)

Un loop para cubrir cada necesidad

- **La especificación:**
  - se valida en cada respuesta y se revisa hasta que nadie la tumba
- **El proyecto:**
  - se itera hasta cumplir la condición de éxito
- **La metodología:**
  - se repite hasta que el caso de uso funciona, en verde

¿Quién cierra el loop?

- **El arnés**: un hook Stop que ejecuta los tests
- **Otro modelo**: /goal
- **El código**: claude -p o el Agent SDK
- **Con su ritmo**: /loop y las tareas programadas
- **El usuario**: un revisor que levanta la mano

Anthropic, documentación de Claude Code (2026)

Agente

Fresh eyes

Tras cada hito,
un revisor sin contexto lee el resultado
(en este caso, la especificación)

Como alguien que se incorpora hoy
Qué no se entiende,
Qué se contradice,
Qué falta

Agente

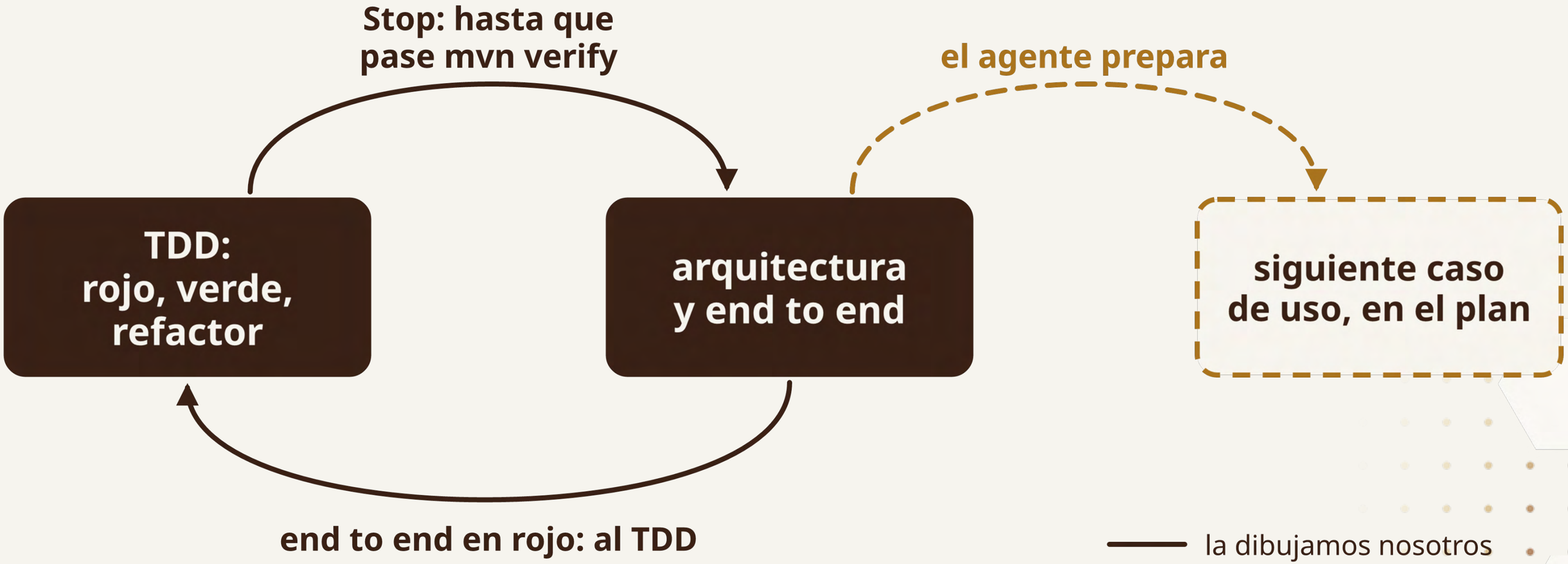
Devil's advocate

Antes de confirmar una decisión, este agente intenta tumbarla

Confirmada, herida o refutada

Su trabajo es intentar quitarnos la razón

La cadena de loops es el grafo



2026

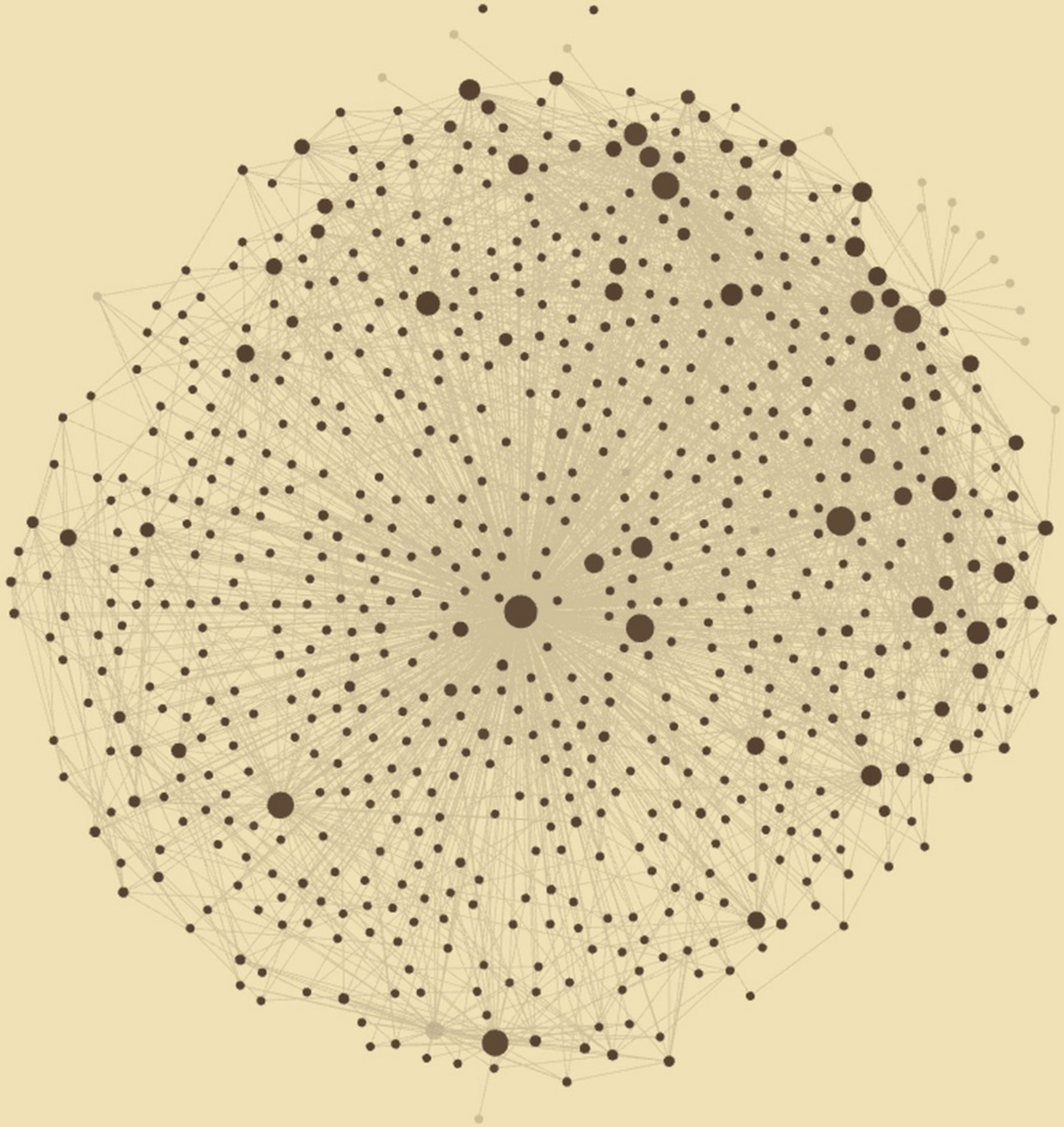
El agente también crea loops

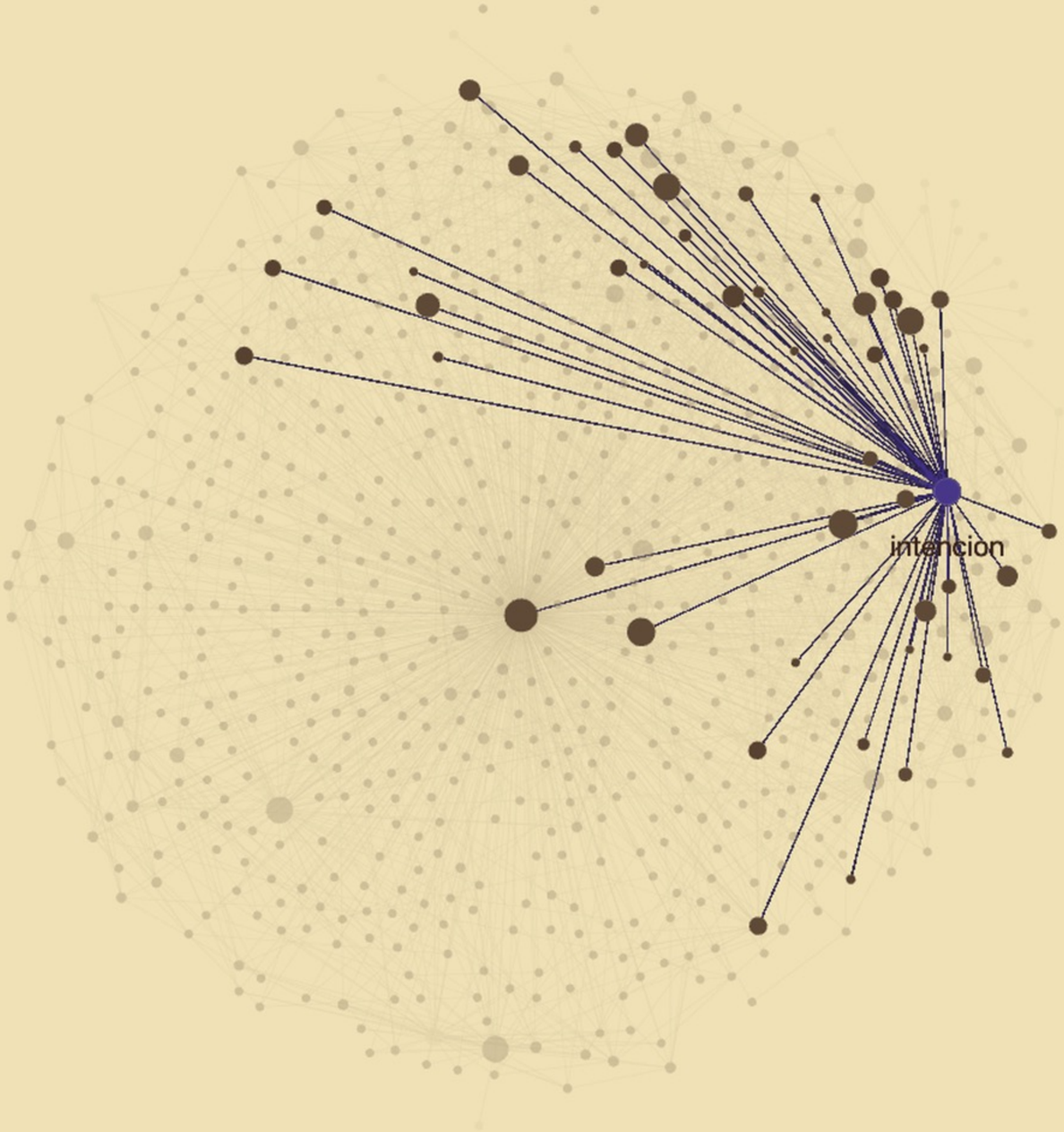
«Claude writes the script for the task you describe»

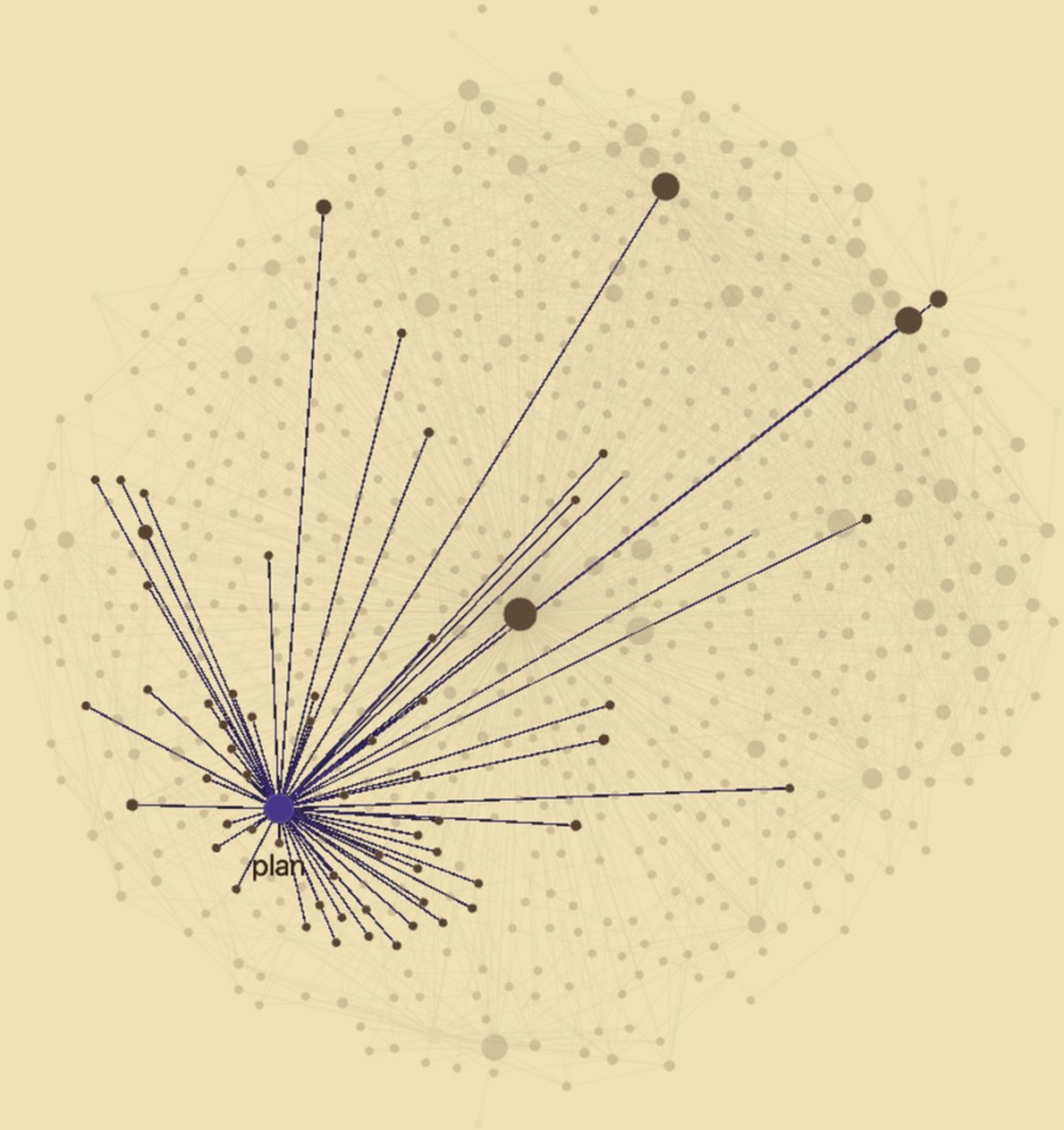
Con subagentes: decide Claude, turno a turno
Con un workflow: decide el script, que escribe Claude
...cuándo está hecho, se lo decimos nosotros

Anthropic, Claude Code, Workflows (2026)

Esta presentación como demostración

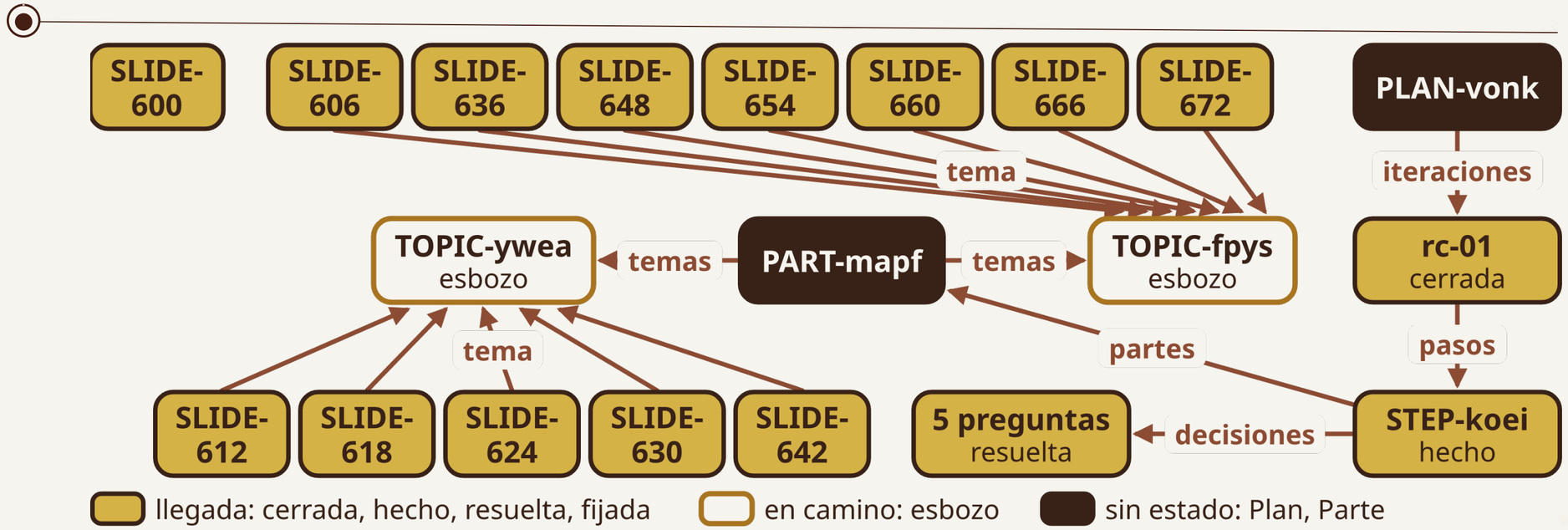



intencion


plan

Demo

De la foto al modelo



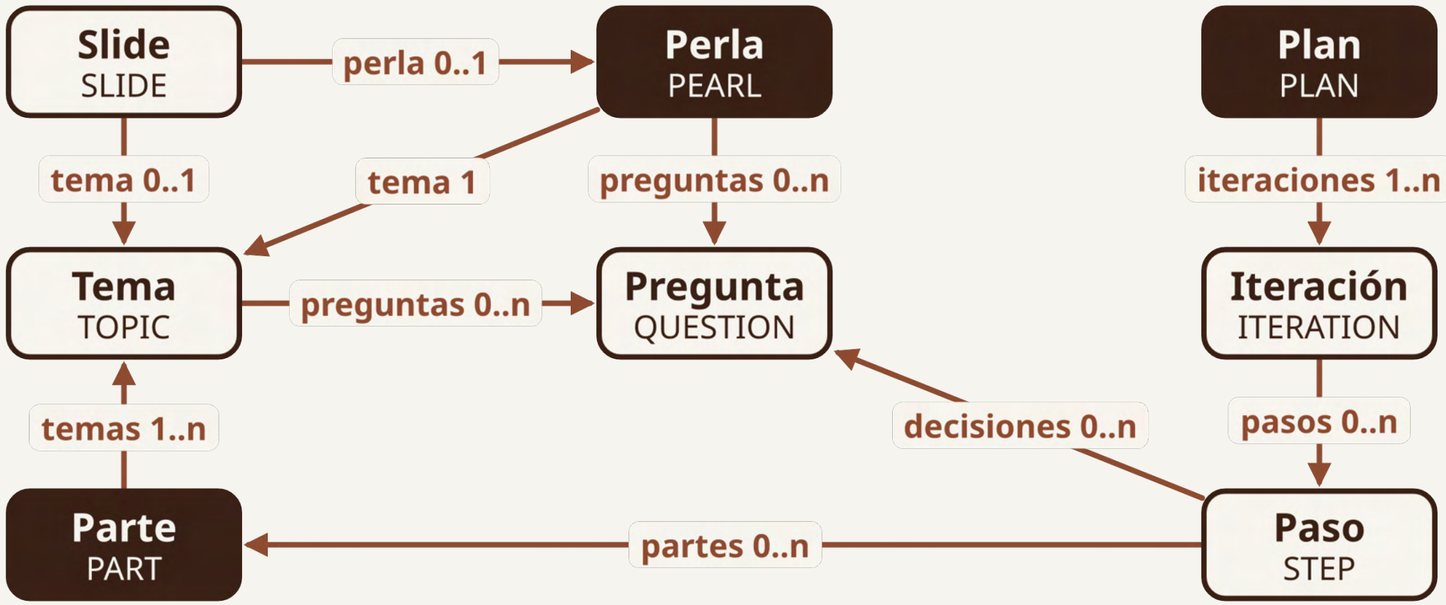
Sin arista: la SLIDE-600, que no tiene tema.
Ningún tema de la parte lleva preguntas en su cabecera.

Esta es una parte del modelo
Cada flecha, una relación de la cabecera del .md

generado desde ontologia.md y las cabeceras del corpus

Demo

El esquema de la ontología



Clases y relaciones

(lo que permite reconstruir el conocimiento)

generado desde ontologia.md y las cabeceras del corpus

Demo

El esquema y una instancia



El esquema a la izquierda
La SLIDE-630, a la derecha

generado desde ontologia.md y las cabeceras del corpus

Demo

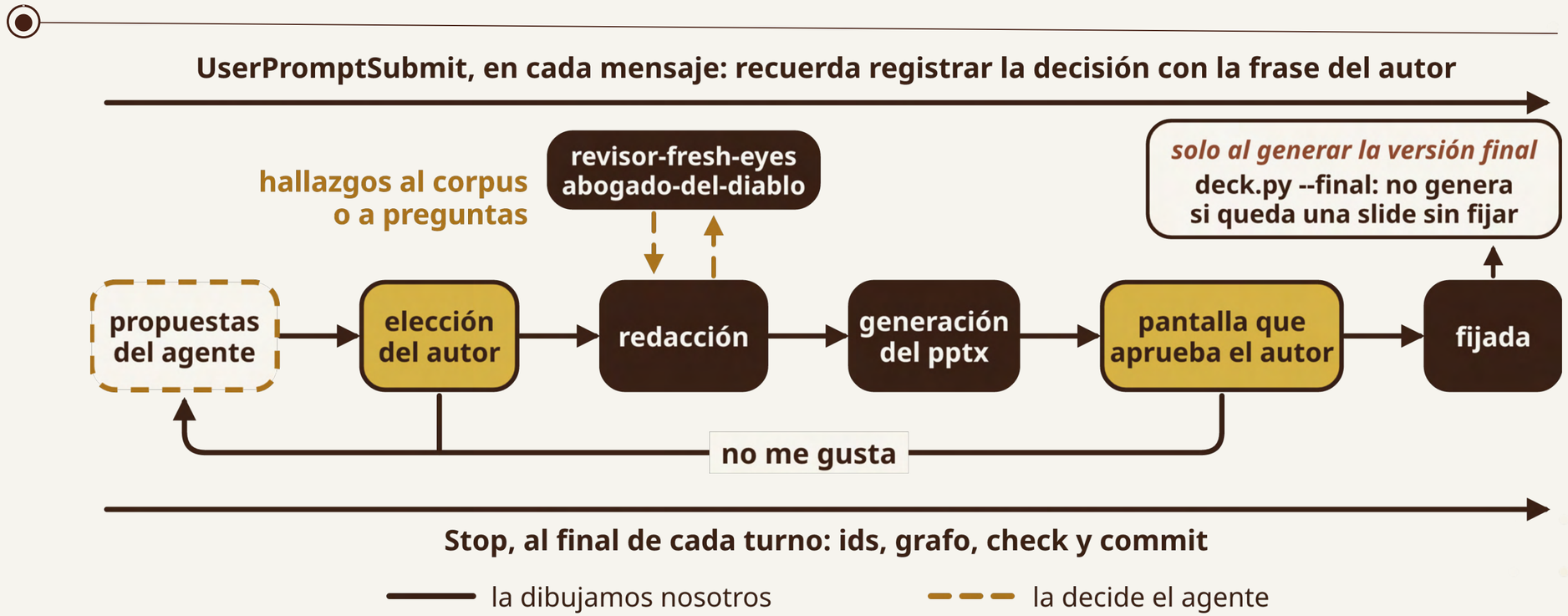
Una máquina de estados

Nosotros la definimos
El agente la recorre
Nosotros la cerramos

slide: borrador → fijada, cuando apruebo su pantalla
pregunta: abierta → resuelta, con mi frase
iteración: en curso → cerrada, cuando pido el tag

Demo

Loop it till you make it

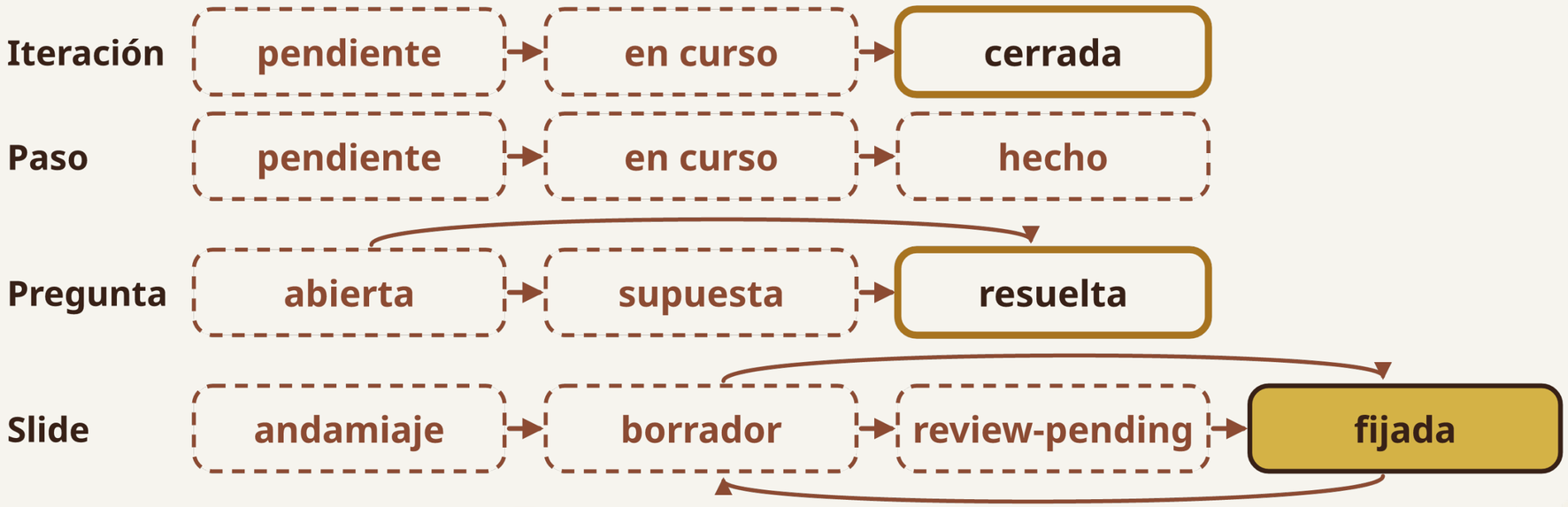


Las quality gates, entre fases

Los hooks, en cada turno

Demo

¿Qué se comprueba?



para: deck.py --final no genera · check da error o avisa, y el commit se hace igual · nada lo comprueba

Fuera de su lista de valores, cualquier estado es un error de check.

Ni mi memoria ni la del agente: el código

anclado a la ontología y al código de check y deck.py

Demo

La vida de una regla

Dentro del turno: check marca el error y el agente lo cambia

```
● Se me ha colado el propio carácter "…" en la hipótesis 2, y check lo marca como error. Lo cambio
```

Reproducción de ese check, sin los cinco avisos de siempre

```
ERROR
ai-first-work/especificacion/preguntas/plan-de-la-parte-de-esta-presentacion-como-demo.md:18:
carácter U+2026: usa tres puntos
...
1 errores, 5 avisos
```

reproducción

Estaba en la memoria y falló
Ahora está en el check y se valida siempre

Captura de Claude Code trabajando en este repo: reproducción de tools/spec.py check

Demo

El trigger tras mi decisión

Registra mi decisión y lanza dos agentes

```
● He registrado tu decisión, con tu frase literal, en TOPIC-jkex, dentro del apartado de la 216. Hay dos agentes trabajando
  en paralelo:

  - modelador-corpus: corrige model/index.md, que pasa de "OKF v0.1" a la v0.2 y al repositorio vigente, y busca otras
    menciones desfasadas en model/ y tools/.
  - arquitecto-conferencia: crea el tema de OKF (fundamentos, utilidad y límites) y lo enlaza en las dos direcciones con
    preguntas, partes, slides, temas, perlas, fuentes y el grafo, no solo con la 216.

  Cuando terminen, compruebo el resultado con spec.py check y te informo.
```

Propone la slide y reparte el trabajo

```
La slide, con la plantilla ribbon-imagen-ancha y ribbon "Demo":

- encabezado: "Lo que ve el modelo";
- afirmación: "23 enlaces escritos" / "7 en la cabecera".

El reparto del trabajo:

- el modelador-corpus hace la función nueva en grafo_ontologia.py;
- el arquitecto-conferencia redacta la 670 en borrador, ajusta la frase de entrada de la 672 y apunta en PART-mapf el
  coste de minutos;
- la skill regenerar-pptx regenera y revisa visualmente la rc-01.
```

Yo decido y delego: los agentes ejecutan *(dentro del arnés)*

Capturas de Claude Code trabajando en este repo, 21/10/2026

Algunos agentes de esta conferencia

`arquitecto-conferencia` **Edita**	Qué se cuenta y dónde: cada idea en un solo sitio y el hilo para 45 minutos
`modelador-corpus` **Edita**	Cómo se guarda: ontología, ids, relaciones, plantillas y el validador
`verificador-web` **Edita solo fuentes/**	Comprueba en el original autor, título, fecha y naturaleza de cada fuente
`revisor-fresh-eyes` **Solo lectura**	Entra sin contexto y reporta fricción, contradicciones, duplicidades y ausencias
`abogado-del-diablo` **Solo lectura**	Ataca cada decisión y da veredicto: confirmada, herida o refutada

.claude/agents/

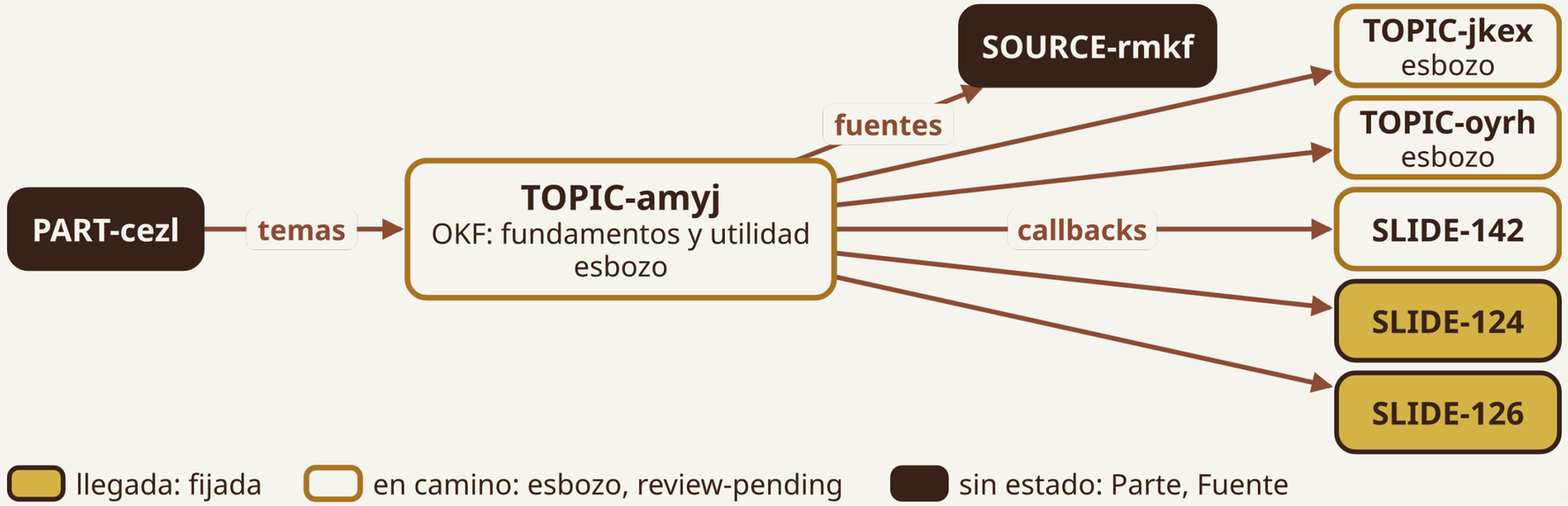
Algunas skills de esta conferencia

`investigar-tema` **La sigue verificador-web**	Investiga en la web y deja fuentes y afirmaciones con su veredicto
`especificar-conferencia` **La sigue arquitecto-conferencia**	Traslada el material a temas, perlas y fuentes sin duplicados, y propone el hilo
`modelar-corpus` **La sigue modelador-corpus**	Cambia el modelo: clases, relaciones, plantillas y spec.py, y migra el corpus
`regenerar-pptx` **La sigue la sesión principal**	Genera los .pptx desde notas, plantillas y .potx, y los revisa en PowerPoint

.claude/skills/

Demo

Lo que ve el modelo



Su cabecera declara 6 relaciones; su texto enlaza 20 conceptos.
14 de ellos no están en la cabecera: para el modelo, no son relaciones.

20 conceptos enlazados en el texto
6 relaciones en la cabecera

generado desde ontologia.md y las cabeceras del corpus

Conclusión


EL PUERTO
LA CIUDAD

Cierre

Complejidad esencial y accidental

No hay balas de plata

Escribir: lo absorbe la IA
Delegar: hay que industrializarlo
Decidir qué construir: sigue siendo nuestro

Frederick P. Brooks, Jr., No Silver Bullet: Essence and Accidents of Software Engineering (1986)

Nos pagan por modelar

Nos pagan por modelar

Nos pagan por construir

Nos pagan por construir

Nos pagan por aportar criterio

Nos pagan por aportar criterio

Nos pagan por decidir

¿A quién seguir?

Andrej Karpathy
Boris Cherny
Steve Yegge
Geoffrey Huntley
Peter Steinberger
Kent Beck
Birgitta Böckeler

¡Gracias!

Es el turno de preguntas y debate

Hemos llegado al estado final del grafo ;)

linkedin.com/in/robertosocrates

